

End-to-End E-Commerce

Data Platform

Project Documentation

Version 1.0

June 7, 2026

Table of Contents

Table of Contents	2
1. Problem Statement.....	6
1.1 Overview.....	6
1.2 Data Sources	6
1.3 Project Objectives.....	6
2. Introduction	7
2.1 Main Components.....	7
2.2 Data Flow Overview.....	8
2.3 Expected Outcomes.....	8
3. Architecture Overview	9
3.1 Overview.....	9
3.2 Architecture Components.....	9
3.3 Data Flow	9
Transactional Pipeline	9
Behavioural Pipeline	9
3.4 Architectural Principles	10
4. Docker Infrastructure	11
4.1 Services.....	11
Workflow Orchestration	11
Messaging & Streaming	11
Databases.....	11
Reporting & Visualisation	11
5. CSV to PostgreSQL Pipeline	12
5.1 Overview.....	12
5.2 Pipeline Flow	12
5.3 Airflow Workflow	12
5.4 Data Cleaning	12
5.5 PostgreSQL Storage.....	13
5.6 Incremental Loading	13
5.7 Key Benefits	13
6. Behavioural JSON to MongoDB Pipeline.....	14
6.1 Overview.....	14
6.2 Pipeline Flow	14
6.3 Behavioural Events.....	14
6.4 Airflow Workflow	14
6.5 Data Validation and Transformation.....	15
6.6 MongoDB Storage Model.....	15
6.7 Key Benefits	15

7. Kafka & Schema Registry.....	16
7.1 Overview.....	16
7.2 Why Kafka?	16
7.3 Topic Organisation.....	16
Transactional Topics	16
Behavioural Topics.....	16
7.4 Schema Registry	17
Schema Strategy.....	17
7.5 Message Flow.....	17
7.6 Kafka Management Tools	18
7.7 Key Benefits	18
8. PostgreSQL to Kafka Pipeline	19
8.1 Overview.....	19
8.2 Pipeline Flow	19
8.3 Source Tables and Topic Mapping.....	19
8.4 Airflow Workflow	19
8.5 Incremental Publishing.....	19
8.6 Batch Processing Strategy.....	20
8.7 Schema Validation	20
8.8 Kafka Message Structure.....	20
8.9 Reliability Considerations.....	20
8.10 Key Benefits.....	20
9. MongoDB to Kafka Pipeline.....	21
9.1 Overview.....	21
9.2 Pipeline Flow	21
9.3 Airflow Workflow	21
9.4 Incremental Publishing.....	21
9.5 Schema Validation.....	21
9.6 Event-Driven Design	22
9.7 Key Benefits	22
10. ClickHouse Integration	23
10.1 Overview.....	23
10.2 Data Ingestion Strategy	23
10.3 Transactional Data Integration	24
10.4 Behavioural Data Integration.....	24
10.5 Materialized Views	24
10.6 Analytical Storage.....	24
10.7 Reporting Foundation	24
10.8 Key Benefits.....	25

11. Reports & Analytics	26
11.1 Overview.....	26
11.2 Reporting Architecture	26
11.3 Report Categories.....	26
Business Performance Reports.....	26
Behavioural Analytics Reports.....	26
11.4 Dashboarding with Metabase.....	26
11.5 Key Benefits.....	27
12. Data Quality & Validation.....	28
12.1 Overview.....	28
12.2 Data Quality Strategy.....	28
12.3 Transactional Data Validation	28
12.4 Behavioural Data Validation.....	29
12.5 Schema Validation	29
12.6 Duplicate Prevention.....	29
12.7 Key Benefits.....	29
13. Challenges & Design Decisions.....	30
13.1 Overview.....	30
13.2 Challenge 1: Managing Multiple Data Sources.....	30
Design Decision	30
13.3 Challenge 2: Preventing Duplicate Processing.....	30
Design Decision	30
13.4 Challenge 3: Data Validation and Consistency.....	30
Design Decision	30
13.5 Challenge 4: Decoupling Operational and Analytical Systems	31
Design Decision	31
13.6 Challenge 5: Schema Management.....	31
Design Decision	31
13.7 Challenge 6: Choosing the Right Storage Technologies	31
Design Decision	31
13.8 Challenge 7: Building a Scalable Analytics Layer.....	31
13.9 Challenge 8: Workflow Automation	32
Design Decision	32
13.10 Lessons Learned	32
13.11 Conclusion	32
14. How to Run the Project.....	33
14.1 Overview.....	33
14.2 Prerequisites.....	33
14.3 Step 1 — Clone the Repository.....	33

14.4 Step 2 — Start the Infrastructure.....	33
14.5 Step 3 — Initialise PostgreSQL Schema.....	33
14.6 Step 4 — Load Transactional Data	34
14.7 Step 5 — Process Behavioural Data	34
14.8 Step 6 — Publish Transactional Data to Kafka.....	34
14.9 Step 7 — Publish Behavioural Data to Kafka	34
14.10 Step 8 — Initialise ClickHouse	34
14.11 Step 9 — Validate Data Flow	34
14.12 Step 10 — Access Dashboards	34

1. Problem Statement

1.1 Overview

Modern e-commerce platforms generate data from multiple sources, including transactional records and user behavioural events. These datasets are often stored in different systems and formats, which makes real-time analytics and reporting complex to implement reliably.

The goal of this project is to design and implement an end-to-end data platform capable of ingesting, validating, processing, streaming, and analysing both transactional and behavioural data.

The platform integrates batch and event-driven processing techniques to create a unified analytics environment that supports reliable reporting and business insights.

1.2 Data Sources

The project processes two distinct categories of data:

- Transactional Data — structured records originating from CSV files (users, products, orders).
- Behavioural Data — semi-structured event records generated by user interactions within the online store.

1.3 Project Objectives

- Build automated data ingestion pipelines for both transactional and behavioural sources.
- Validate and clean incoming data before it reaches operational storage.
- Store operational data in appropriate databases (PostgreSQL and MongoDB).
- Stream data reliably through Apache Kafka.
- Maintain schema consistency using Schema Registry.
- Load processed data into ClickHouse for high-performance analytics.
- Generate business and behavioural reports through Metabase dashboards.

2. Introduction

Data-driven decision-making requires a reliable platform capable of collecting data from multiple sources, processing it efficiently, and making it available for analytical workloads.

In this project, an online store data platform was developed using modern data engineering architecture. The platform combines batch processing, event streaming, schema management, workflow orchestration, and analytical storage to create a complete end-to-end data pipeline.

The system handles two distinct types of data:

- Structured transactional data originating from CSV datasets.
- Semi-structured behavioural events generated by user interactions.

To support these workloads, technologies were selected based on their specific strengths: PostgreSQL stores transactional data, MongoDB stores behavioural events, Kafka serves as the streaming backbone, and ClickHouse provides high-performance analytical storage.

Apache Airflow orchestrates all workflows and automates data movement between components. Schema Registry ensures consistent message formats across the streaming layer, while Metabase provides reporting and dashboard capabilities.

2.1 Main Components

Table 1.Explain each Component Purpose

Component	Purpose
Apache Airflow	Workflow orchestration and pipeline automation
PostgreSQL	Transactional data storage
MongoDB	Behavioural event storage
Apache Kafka	Data streaming platform
Schema Registry	Message schema management
ClickHouse	Analytical database for reporting
Metabase	Reporting and dashboard visualisation

2.2 Data Flow Overview

The platform consists of two independent ingestion pipelines that converge at Kafka and ultimately feed the analytical layer.

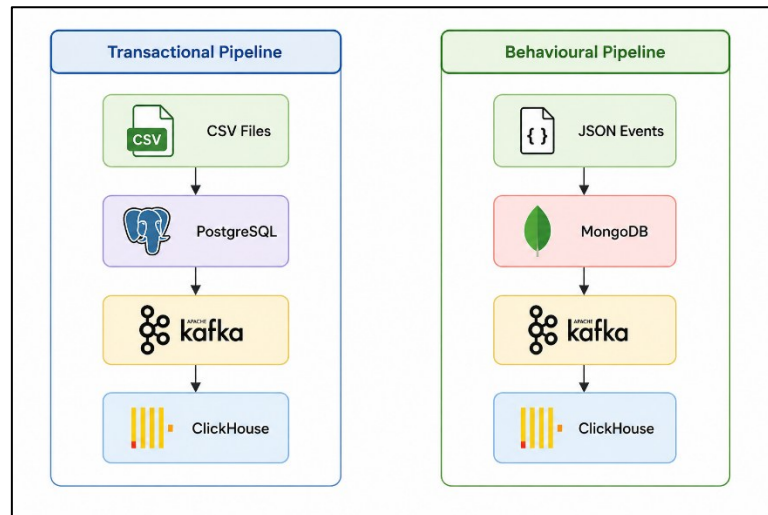


Figure 1. Data Flow Overview

Both pipelines are designed with validation, incremental processing, and monitoring mechanisms to ensure data reliability and consistency.

2.3 Expected Outcomes

By the end of the pipeline, all incoming data is transformed into structured analytical datasets that can be queried efficiently and visualised through dashboards and reports. The platform provides a reliable, scalable foundation for business intelligence.

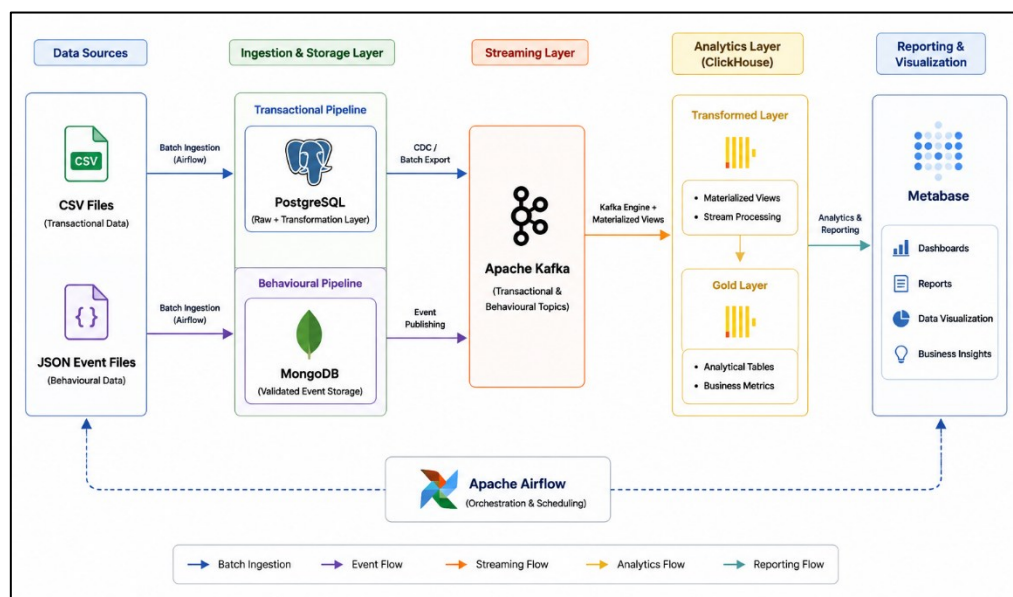


Figure 2. End-to-End Data Pipeline Architecture

3. Architecture Overview

3.1 Overview

The project follows a modern data engineering architecture designed to process both transactional and behavioural data through separate ingestion pipelines while maintaining a unified analytical layer.

Two independent data sources are ingested, validated, and stored in operational databases. The data is then streamed through Kafka and ultimately loaded into ClickHouse for analytics and reporting.

The architecture combines batch processing, event streaming, workflow orchestration, schema management, and analytical storage into a single end-to-end platform.

3.2 Architecture Components

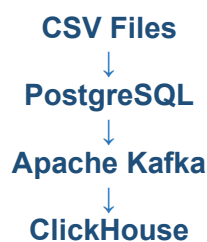
The platform consists of the following main components:

- Apache Airflow — workflow orchestration and pipeline automation.
- PostgreSQL — transactional data storage.
- MongoDB — behavioural event storage.
- Apache Kafka — data streaming platform.
- Schema Registry — message schema management.
- ClickHouse — high-performance analytical database.
- Metabase — reporting and dashboard visualisation.

3.3 Data Flow

The platform contains two main pipelines, both converging on the same analytical destination:

Transactional Pipeline



Behavioural Pipeline



3.4 Architectural Principles

The following design principles guided the platform architecture:

- Separation of operational and analytical workloads.
- Independent processing of transactional and behavioural data.
- Schema-controlled data streaming via Schema Registry.
- Incremental and automated data processing through Airflow.
- Scalable analytical storage using ClickHouse.
- Decoupled system components through Kafka as an intermediary layer.

These principles make the platform easier to maintain, extend, and monitor over time.

4. Docker Infrastructure

All platform components are deployed and managed using Docker Compose. Containerisation provides a consistent, reproducible, and isolated environment for all services involved in the project.

4.1 Services

Workflow Orchestration

- Airflow API Server
- Airflow Scheduler
- Airflow Worker
- Airflow Triggerer
- Airflow DAG processor
- Airflow Init
- Airflow CLI

Messaging & Streaming

- Apache Kafka
- Schema-Registry
- Kafka-Connect

Databases

- PostgreSQL (Business Transactions)
- PostgreSQL (Airflow Metadata)
- MongoDB
- ClickHouse

Reporting & Visualisation

- Metabase
- PostgreSQL (Metabase Metadata)

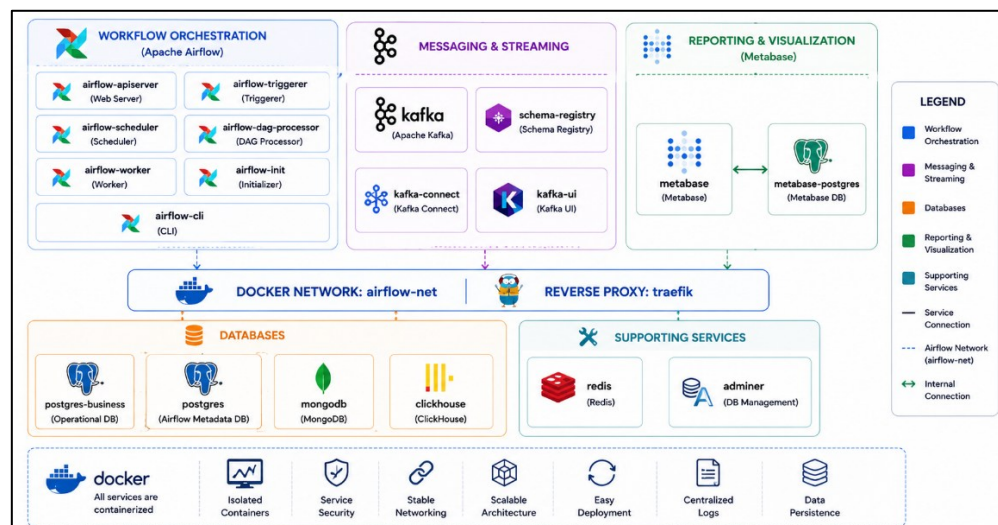


Figure 3. Docker Infrastructure - Overall Architecture

5. CSV to PostgreSQL Pipeline

5.1 Overview

This pipeline is responsible for ingesting transactional business data into PostgreSQL. Three CSV datasets are processed:

- users.csv — customer information
- products.csv — product catalogue
- orders.csv — purchase transactions

Apache Airflow orchestrates the entire workflow and automatically loads new or updated files into the database.

5.2 Pipeline Flow

Table 2. Transactional Data Information

Source File	Description
users.csv	Customer information
products.csv	Product catalogue
orders.csv	Purchase transactions

5.3 Airflow Workflow

The pipeline is implemented using the `csv_etl_pipeline` DAG. The workflow performs the following steps:

- Detect file changes using hash or modification tracking.
- Read CSV data into memory.
- Validate and clean records.
- Load cleaned data into PostgreSQL.
- Update the processing state to avoid redundant reloads.

To avoid unnecessary processing, files are loaded only when a change is detected using Airflow File Sensor.

5.4 Data Cleaning

Before loading data into PostgreSQL, a lightweight transformation layer is applied. Main cleaning operations include:

- Removing empty or null records.
- Standardising column names to a consistent format.
- Converting data types to match the target schema.
- Removing duplicate records.
- Validating business values against accepted rules.

Examples of applied transformations:

- Email addresses are normalised and validated.
- Product categories are standardised against a reference list.
- Payment methods are validated against accepted values.
- Records with invalid or missing key fields are discarded.

5.5 PostgreSQL Storage

The cleaned data is stored in three PostgreSQL tables: users, products, and orders. The database serves as the operational storage layer and acts as the source system for downstream Kafka pipelines.

5.6 Incremental Loading

Instead of reloading all files on every execution, the pipeline tracks previously processed files using Airflow Variables. Only modified files are processed during each run, reducing unnecessary database operations and improving overall efficiency.

5.7 Key Benefits

- Automated and repeatable data ingestion.
- Improved data quality through early-stage validation.
- Incremental processing to minimise redundant operations.
- Centralised transactional storage ready for downstream consumption.
- Native integration with the Kafka streaming layer.

6. Behavioural JSON to MongoDB Pipeline

6.1 Overview

This pipeline is responsible for ingesting and processing behavioural event data generated by user interactions within the online store.

Unlike transactional data, behavioural events are semi-structured and represent user actions such as browsing products, performing searches, adding items to a cart, or completing a purchase. Apache Airflow orchestrates the ingestion process, while MongoDB is used as the storage layer for the processed events.

6.2 Pipeline Flow



The pipeline continuously scans incoming behavioural files, processes new records, and stores them in MongoDB collections.

6.3 Behavioural Events

The platform supports multiple event types representing different stages of the customer journey:

- Page View
- Product Search
- Add to Cart
- Remove from Cart
- Cart View
- Wishlist Add
- Checkout Start
- Payment Attempt
- Order Complete
- Review Submit

Each event captures specific information about user behaviour and contributes to a more complete understanding of the customer journey.

6.4 Airflow Workflow

The ingestion process is implemented using dedicated Airflow DAGs responsible for:

- Detecting incoming behavioural files.
- Identifying previously processed files to avoid duplication.
- Validating event records against required field definitions.

- Transforming raw JSON data into the target storage format.
- Loading valid records into the appropriate MongoDB collections.

6.5 Data Validation and Transformation

Before loading data into MongoDB, each event is validated to ensure that required fields are present. The pipeline verifies the following fields for all event types:

- Timestamp
- User ID
- Session ID
- Event Type
- Device

Additional validation rules are applied depending on the specific event type. Invalid records are skipped and logged without interrupting the processing of valid data, maintaining pipeline reliability without sacrificing data quality.

6.6 MongoDB Storage Model

MongoDB was selected because behavioural events are semi-structured and vary in their payload depending on the event type. Rather than storing all events in a single collection, the pipeline organises them into separate collections by event type:

- page_view
- product_search
- add_to_cart
- checkout_start
- payment_attempt
- order_complete

This structure simplifies querying, debugging, and future analytical processing.

6.7 Key Benefits

- Automated ingestion of behavioural event data.
- Robust validation of incoming events.
- Prevention of duplicate file processing.
- Flexible storage for semi-structured data.
- Clear separation of event types across MongoDB collections.
- Foundation for behavioural analytics and reporting.

7. Kafka & Schema Registry

7.1 Overview

Apache Kafka serves as the central data streaming platform and acts as the bridge between operational databases and the analytical layer. After data is processed and stored in PostgreSQL and MongoDB, it is published to Kafka topics and made available to downstream consumers such as ClickHouse.

To ensure data consistency and enforce message structures, Schema Registry is used alongside Kafka to manage schemas and validate messages before publication.

7.2 Why Kafka?

The project contains multiple independent systems — PostgreSQL, MongoDB, ClickHouse, and reporting tools. Direct communication between these systems would create tight coupling and make future changes more difficult.

Kafka introduces a streaming intermediary layer between producers and consumers, allowing systems to exchange data asynchronously while remaining independent of one another.

Key benefits of using Kafka in this architecture:

- Decouples producers and consumers.
- Enables scalable and reliable data movement.
- Supports both real-time and batch-oriented processing.
- Improves system flexibility and future extensibility.
- Simplifies integration of additional downstream consumers.

7.3 Topic Organisation

The project separates data into multiple topics based on business entities and event types.

Transactional Topics

These topics contain structured business data originating from PostgreSQL:

- postgres.users
- postgres.products
- postgres.orders

Behavioural Topics

These topics contain user interaction events originating from MongoDB:

- events.page_view
- events.search
- events.add_to_cart
- events.remove_from_cart
- events.cart_view
- events.checkout
- events.payment

- events.order
- events.review
- events.wishlist

7.4 Schema Registry

Schema Registry manages the structure of Kafka messages. Instead of allowing producers to send arbitrary payloads, every message must conform to a predefined schema. This creates a clear data contract between producers and consumers.

Schema Strategy

The project uses two schema approaches depending on the data source:

- Transactional Data (users, products, orders) — published using JSON Schema, which is well-suited to the relatively simple and predictable structure of transactional records.
- Behavioural Data (page views, searches, checkout events, etc.) — published using Avro Schema, which is better suited to nested structures and varying payloads, and provides a more compact serialisation format.

7.5 Message Flow

Before a message is written to Kafka, its structure is validated against the registered schema. Only valid messages are accepted and published to the topic. Invalid messages are rejected during the validation process and prevented from entering the Kafka topic. These messages are logged with detailed error information, enabling operators to investigate schema violations and maintain data consistency across the platform.

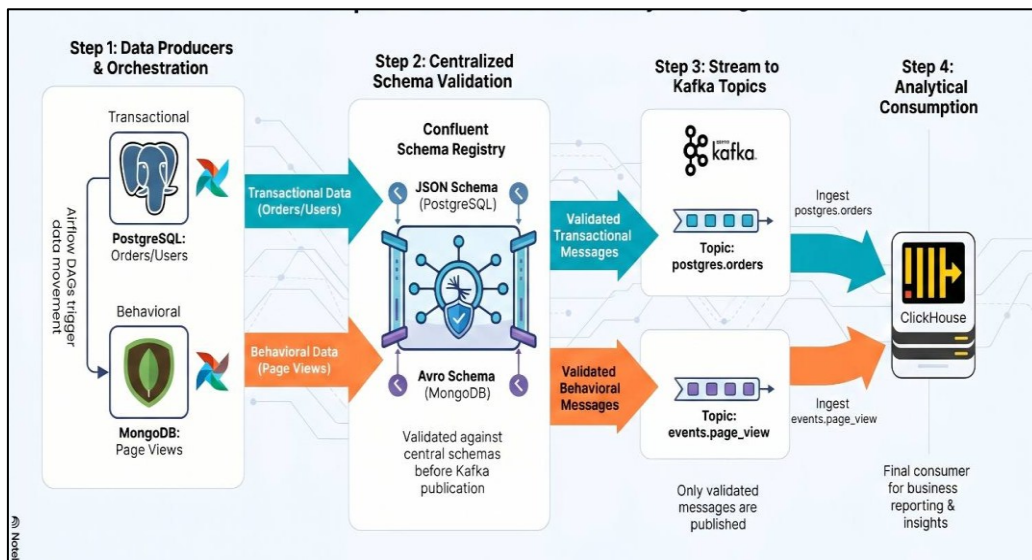


Figure 4. Data Message Flow

7.6 Kafka Management Tools

To simplify development and debugging, the project includes Kafka UI, which provides:

- Topic inspection and browsing.
- Message content viewer.
- Consumer group monitoring.
- Topic statistics and partition details.
- Schema verification interface.

7.7 Key Benefits

- Reliable data transportation between independent systems.
- Clear separation of operational and analytical layers.
- Schema-controlled message exchange with enforcement at publish time.
- Extensible foundation for future consumers and integrations.
- Support for near real-time analytics.

8. PostgreSQL to Kafka Pipeline

8.1 Overview

This pipeline is responsible for publishing transactional data from PostgreSQL to Kafka. After transactional datasets are cleaned and stored in PostgreSQL, they are transferred to Kafka topics where they become available for downstream consumers such as ClickHouse.

The pipeline is implemented as an Airflow DAG and follows an incremental publishing strategy to avoid sending duplicate records.

8.2 Pipeline Flow



8.3 Source Tables and Topic Mapping

Table 3. Transaction Data Topics

PostgreSQL Table	Kafka Topic
users	postgres.users
products	postgres.products
orders	postgres.orders

8.4 Airflow Workflow

The publishing process is implemented through the `postgres_to_kafka_batch_publish` DAG. During execution, the DAG:

- Connects to PostgreSQL and reads new records since the last processed ID.
- Validates data against the registered JSON Schema.
- Publishes valid messages to the corresponding Kafka topics.
- Updates the processing state (last processed record ID) for the next run.

8.5 Incremental Publishing

To avoid republishing records that have already been sent to Kafka, the pipeline maintains a processing state using Airflow Variables:

- `postgres_to_kafka_users_last_id`

- postgres_to_kafka_products_last_id
- postgres_to_kafka_orders_last_id

During each execution, the last processed record ID is retrieved, only newer records are selected and published, and the latest processed ID is then stored for the next run.

8.6 Batch Processing Strategy

Records are fetched from PostgreSQL in batches rather than one at a time. This approach provides:

- Reduced database load through fewer queries.
- Better throughput for high-volume publishing.
- Improved Kafka producer performance.
- Efficient handling of large datasets as data volumes grow.

8.7 Schema Validation

Before being published to Kafka, transactional records are validated using Schema Registry with JSON Schema. Schemas are defined for users, products, and orders. This ensures that all published messages follow a predictable structure that downstream systems can safely consume.

8.8 Kafka Message Structure

Each Kafka message contains a complete representation of a business entity. Example user message:

```
{ "user_id": "u123", "name": "John Doe", "email":  
  "john@example.com", "loyalty_tier": "Gold" }
```

8.9 Reliability Considerations

The Kafka producer is configured to prioritise delivery reliability through:

- Message acknowledgements from Kafka brokers.
- Retry mechanisms for transient failures.
- State tracking to resume from the last successful point.
- Controlled batch publishing to prevent overloading the broker.

8.10 Key Benefits

- Automated and repeatable data publishing.
- Incremental processing to prevent duplicate records.
- Schema-controlled messaging with enforcement at publish time.
- Decoupling of operational and analytical system workloads.
- Scalable data movement through Kafka.

9. MongoDB to Kafka Pipeline

9.1 Overview

This pipeline is responsible for publishing behavioural event data from MongoDB to Kafka. After events are validated and stored in MongoDB, they are transferred to Kafka topics for downstream analytical processing.

Unlike transactional data, behavioural events represent user interactions and are naturally event-driven. Each event type is published to its own dedicated Kafka topic, allowing consumers to independently process specific user activities.

9.2 Pipeline Flow



9.3 Airflow Workflow

The publishing process is implemented through the `mongo_to_kafka_batch_publish` DAG. During execution, the pipeline:

- Connects to MongoDB and reads newly inserted events.
- Validates events against the registered Avro schemas.
- Publishes valid messages to the corresponding Kafka topics.
- Updates the processing state (last processed ObjectId) for each collection.

9.4 Incremental Publishing

To prevent duplicate event delivery, the pipeline tracks the latest processed MongoDB ObjectId per event type using Airflow Variables:

- `mongo_to_kafka_page_view_last_id`
- `mongo_to_kafka_product_search_last_id`
- `mongo_to_kafka_checkout_start_last_id`

During each run, only documents with a newer ObjectId than the stored value are selected, ensuring every event is processed exactly once.

9.5 Schema Validation

Behavioural events are validated using Avro schemas via Schema Registry before publication. Avro was selected because many events contain nested structures and event-specific attributes, such as:

- Cart items and quantities.
- Payment information and method details.

- Search terms and result metadata.
- Checkout step and funnel stage.

9.6 Event-Driven Design

Unlike transactional entities, behavioural records represent user actions at a point in time. Publishing these actions as individual events enables detailed behavioural analytics and customer journey analysis, including:

- Product search and discovery patterns.
- Cart abandonment analysis.
- Payment success and failure rates.
- Review and sentiment tracking.

9.7 Key Benefits

- Automated and repeatable event publishing.
- Incremental processing with ObjectId-based state management.
- Event-driven architecture aligned with user behaviour patterns.
- Schema-controlled messaging using Avro.
- Foundation for customer behaviour analytics and funnel reporting.

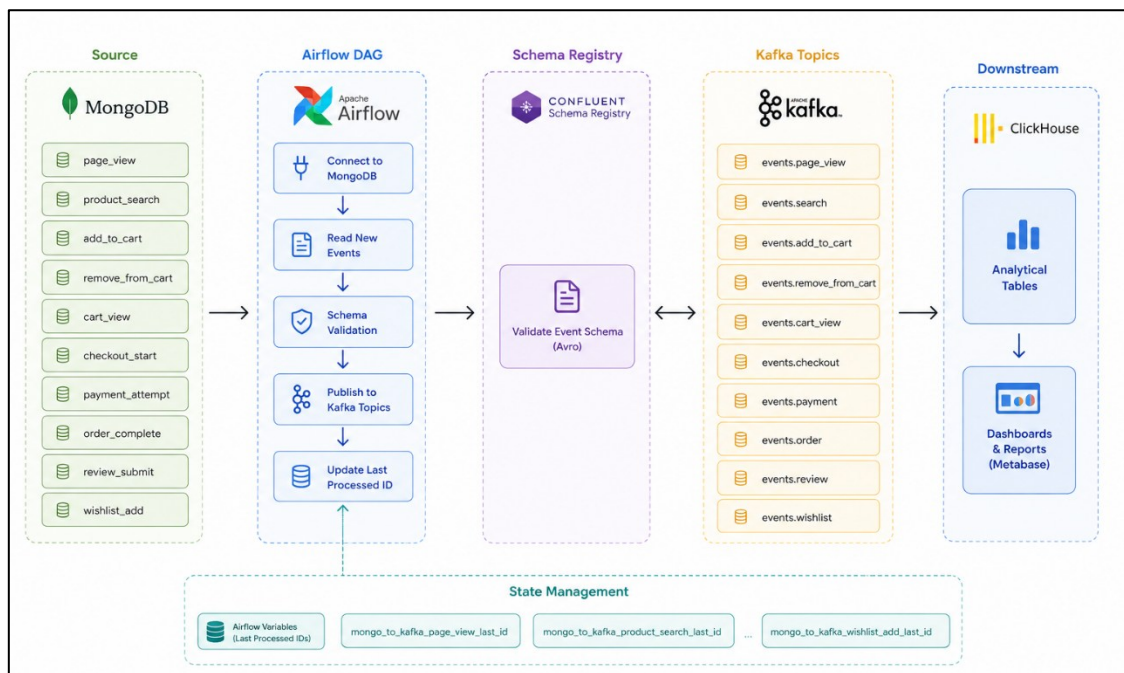


Figure 5. ClickHouse Data Ingestion Flow

10. ClickHouse Integration

10.1 Overview

ClickHouse serves as the analytical layer of the platform and is responsible for transforming streamed data into queryable datasets optimised for reporting and business analytics.

While PostgreSQL and MongoDB act as operational storage systems, ClickHouse is designed specifically for analytical workloads. It continuously consumes data from Kafka and stores it in column-oriented structures optimised for aggregations, filtering, and large-scale reporting. The result is a centralised analytics platform capable of supporting business intelligence tools such as Metabase.

10.2 Data Ingestion Strategy

Instead of loading data directly from operational databases, ClickHouse consumes messages from Kafka topics. The ingestion process follows this pattern:



Figure 6. ClickHouse to Metabase Flow

This architecture enables continuous ingestion while keeping the final analytical tables optimised for querying.

10.3 Transactional Data Integration

Transactional data originating from PostgreSQL is consumed from the following Kafka topics:

- postgres.users
- postgres.products
- postgres.orders

For each topic, ClickHouse uses a Kafka Engine table, a Materialized View, and a final MergeTree table. This three-layer structure enables automatic ingestion and transformation of incoming messages.

10.4 Behavioural Data Integration

Behavioural events follow the same architecture. Each event type is stored independently, making it easier to analyse user behaviour and customer journeys:

- events.page_view
- events.search
- events.checkout
- events.payment
- events.order

This design also improves query performance because event datasets remain logically separated.

10.5 Materialized Views

Materialized Views play a central role in the ingestion process. They are responsible for:

- Reading incoming messages from Kafka Engine tables.
- Transforming and extracting required fields.
- Loading processed records into the final MergeTree analytical tables.

Because Materialized Views operate automatically, no additional ingestion jobs are required once the pipeline is running.

10.6 Analytical Storage

The final ClickHouse tables are designed for reporting and analytical queries, providing:

- Fast aggregations over large datasets.
- Efficient filtering with column-oriented storage.
- High compression ratios to reduce storage overhead.
- Optimised query performance for business intelligence workloads.

10.7 Reporting Foundation

The data stored in ClickHouse forms the foundation for all business reports and KPIs, including:

- Sales performance and revenue analysis.
- Customer purchasing behaviour.

- Product performance metrics.
- Conversion funnel analysis.
- Behavioural event analytics and journey mapping.

10.8 Key Benefits

- Near real-time analytical ingestion via Kafka.
- Automated consumption without manual ingestion jobs.
- High-performance reporting suitable for large data volumes.
- Scalable architecture — new topics and tables can be added without modifying upstream systems.
- Complete decoupling of reporting workloads from operational databases.
- Direct integration with Metabase for dashboarding.

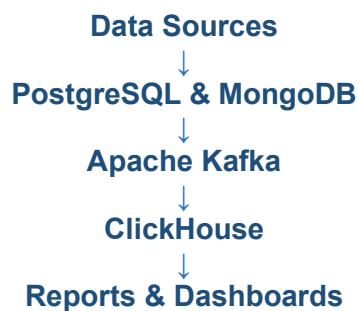
11. Reports & Analytics

11.1 Overview

The ultimate goal of any data platform is to transform raw data into meaningful business insights. After transactional and behavioural data have been ingested, validated, streamed through Kafka, and stored in ClickHouse, they become available for analytical reporting.

This layer enables stakeholders to monitor business performance, understand customer behaviour, and make data-driven decisions based on reliable and up-to-date information.

11.2 Reporting Architecture



This architecture ensures that reporting workloads remain completely isolated from operational systems, preventing analytical queries from impacting the performance of transactional databases.

11.3 Report Categories

Business Performance Reports

Reports focused on sales, orders, products, and customer activity:

- Revenue analysis and trends.
- Order volume and fulfilment tracking.
- Product performance and inventory metrics.
- Customer activity and loyalty analysis.

Behavioural Analytics Reports

Reports focused on user interactions and customer journeys:

- Search behaviour and discovery patterns.
- Product engagement and wishlist activity.
- Checkout activity and cart abandonment rates.
- Conversion funnel analysis.

11.4 Dashboarding with Metabase

The analytical datasets stored in ClickHouse are visualised using Metabase dashboards, providing a user-friendly interface for:

- Business performance monitoring.

- KPI tracking and goal measurement.
- Trend analysis and time-series reporting.
- Operational dashboards accessible to non-technical stakeholders.

This allows business users to explore insights without writing SQL queries directly against the database.

11.5 Key Benefits

- Centralised reporting layer built on ClickHouse.
- Fast analytical queries across large datasets.
- Near real-time data availability after Kafka ingestion.
- Business-oriented KPIs and dashboards via Metabase.
- Complete separation of analytics from operational system workloads.

12. Data Quality & Validation

12.1 Overview

Data quality is a critical aspect of any data engineering project. Poor-quality data leads to inaccurate reports, unreliable analytics, and incorrect business decisions. To ensure consistency and reliability across the platform, validation and cleaning mechanisms are applied at multiple stages of the pipeline before data reaches the analytical layer.

12.2 Data Quality Strategy

Validation is performed independently within each pipeline according to the type of data being processed, following this lifecycle:

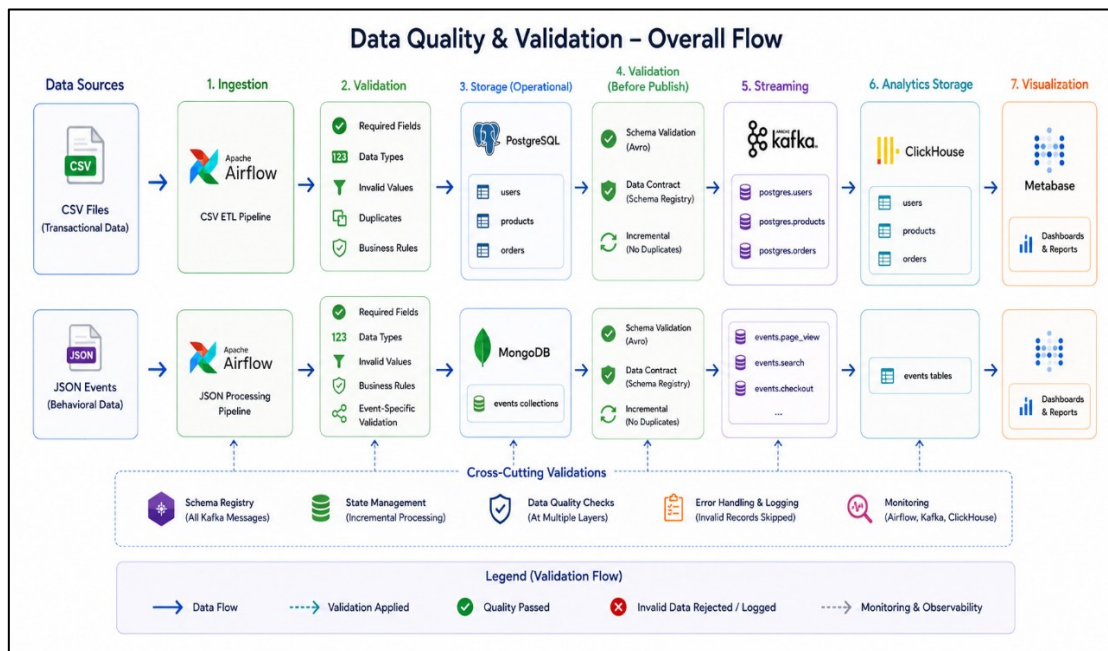


Figure 7. Data Quality & Validation

12.3 Transactional Data Validation

Transactional datasets are validated before being loaded into PostgreSQL. Key validation checks include:

- Required field validation — records missing primary keys are rejected.
- Data type conversion — fields are converted to match the target schema.
- Duplicate removal — records already present in the database are filtered out.
- Invalid value filtering — unsupported categories and payment methods are removed.
- Business rule validation — values are checked against known reference lists.

12.4 Behavioural Data Validation

Behavioural events are validated before being stored in MongoDB. Each event must contain the following mandatory fields:

- Timestamp
- User ID
- Session ID
- Event Type
- Device

In addition, event-specific fields are validated according to the event type. For example, payment events require payment details, and review events require a rating value. Invalid events are skipped and logged without interrupting the processing of valid records.

12.5 Schema Validation

Schema Registry introduces an additional quality control layer for Kafka messages. Before data is published to Kafka, it must conform to the registered schema. This provides:

- Consistent message structures across all producers.
- Reduced integration errors between systems.
- Safer producer-to-consumer communication.
- Improved maintainability as schemas evolve over time.

12.6 Duplicate Prevention

The project includes multiple mechanisms to prevent duplicate processing:

- File processing state tracking to avoid reprocessing the same input files.
- Incremental publishing from PostgreSQL to Kafka using last-processed record IDs.
- Incremental publishing from MongoDB to Kafka using last-processed ObjectIds.

12.7 Key Benefits

- Improved data consistency across all layers of the platform.
- Reduced reporting errors resulting from invalid source data.
- Protection against invalid, incomplete, or duplicate records.
- Reliable downstream analytics built on clean, trusted data.
- Stronger data governance throughout the entire pipeline.

13. Challenges & Design Decisions

13.1 Overview

Building an end-to-end data platform involves much more than connecting technologies together. Throughout the implementation, several architectural and technical decisions were made to balance simplicity, reliability, scalability, and maintainability. This section documents the most important challenges encountered and the reasoning behind key design choices.

13.2 Challenge 1: Managing Multiple Data Sources

The project processes two fundamentally different types of data — transactional CSV files and semi-structured behavioural JSON events. These datasets differ in structure, storage requirements, and processing patterns.

Design Decision

Rather than forcing both datasets through a single pipeline, separate ingestion paths were implemented: CSV files flow into PostgreSQL, and JSON events flow into MongoDB. This allows each data type to be processed using the technology that best fits its characteristics.

13.3 Challenge 2: Preventing Duplicate Processing

Since pipelines run repeatedly on a schedule, a key challenge was ensuring that data is not processed multiple times. Duplicate processing would lead to duplicate records, incorrect analytics, and increased processing costs.

Design Decision

State management was introduced at multiple layers: file processing state is stored in MongoDB, and Airflow Variables track the last processed record for both the PostgreSQL and MongoDB publishing pipelines. This ensures that only new data is processed during each execution.

13.4 Challenge 3: Data Validation and Consistency

Raw datasets may contain missing values, invalid formats, inconsistent categories, and incorrect data types. Without validation, these issues would propagate throughout the platform and affect report accuracy.

Design Decision

Validation and transformation logic were introduced before data is stored in operational databases. This ensures that downstream systems receive cleaner, more reliable data from the earliest possible stage in the pipeline.

13.5 Challenge 4: Decoupling Operational and Analytical Systems

Running analytical queries directly on operational databases is not scalable and can negatively impact application performance.

Design Decision

Kafka was introduced as an intermediary streaming layer between operational storage and the analytical layer, allowing both workloads to evolve independently without interference.

13.6 Challenge 5: Schema Management

As data moves through Kafka, producers and consumers must agree on a consistent message structure. Without schema control, even minor changes can break downstream consumers unexpectedly.

Design Decision

Schema Registry was integrated into the platform to enforce data contracts and validate messages before publication. This improves reliability and simplifies future schema evolution.

13.7 Challenge 6: Choosing the Right Storage Technologies

A single database is rarely optimal for every type of workload. Transactional data requires strong structure and relational integrity, while behavioural events are more flexible and semi-structured.

Design Decision

A polyglot persistence approach was adopted, selecting storage technologies based on workload characteristics:

Table 4. Design Decision for Storage Technology

Data Type	Storage Technology
Transactional Data	PostgreSQL
Behavioural Events	MongoDB
Analytics & Reporting	ClickHouse

13.8 Challenge 7: Building a Scalable Analytics Layer

Reporting systems require fast aggregations and efficient analytical queries, especially as data volumes grow. Traditional transactional databases are not optimised for this workload. ClickHouse was selected as the analytical database due to its column-oriented

storage, high-performance aggregations, efficient compression, and native Kafka integration. This enables near real-time analytics without affecting operational systems.

13.9 Challenge 8: Workflow Automation

Manually running ingestion and publishing processes would quickly become unmanageable as the number of pipelines grows.

Design Decision

Apache Airflow was adopted as the orchestration layer to automate data ingestion, validation, data movement, and publishing workflows. Airflow also provides visibility into pipeline execution and simplifies operational management.

13.10 Lessons Learned

Several important lessons emerged during the development of this project:

- Data quality should be enforced as early as possible in the pipeline.
- State management is essential for reliable incremental processing.
- Different workloads often require different storage technologies.
- Decoupling systems through Kafka simplifies future expansion and integration.
- Analytical workloads should remain separate from operational workloads.
- Automation through Airflow significantly improves reliability and maintainability.

13.11 Conclusion

The final architecture is the result of a series of design decisions aimed at building a reliable, maintainable, and scalable data platform. While the project was developed in an educational setting, the challenges encountered closely resemble those found in real-world data engineering environments. The solutions adopted throughout the implementation reflect common industry practices for handling ingestion, streaming, storage, validation, and analytics at scale.

14. How to Run the Project

14.1 Overview

This section provides the steps required to deploy and execute the entire data platform. All services are containerised using Docker Compose, allowing the complete environment to be started with a minimal setup process.

The project consists of several independent pipelines that should be executed in a specific order to ensure proper data flow from ingestion through to analytics.

14.2 Prerequisites

Before running the project, ensure the following tools are installed on the host machine:

- Docker
- Docker Compose
- Git

It is also recommended to allocate sufficient system resources, as multiple services run simultaneously — including Airflow, Kafka, MongoDB, PostgreSQL, ClickHouse, and Metabase.

14.3 Step 1 — Clone the Repository

Clone the project repository and navigate to the project directory:

```
git clone <repository-url>
cd online_store_data_pipeline
```

14.4 Step 2 — Start the Infrastructure

Start all services using Docker Compose:

```
docker compose up -d
```

Verify that all containers are running:

```
docker ps
```

At this stage, the following services should be accessible: Airflow, PostgreSQL, MongoDB, Kafka, Schema Registry, Kafka UI, ClickHouse, and Metabase.

14.5 Step 3 — Initialise PostgreSQL Schema

Run the PostgreSQL schema initialisation DAG from Airflow to create the required tables for transactional data.

14.6 Step 4 — Load Transactional Data

Run the CSV ingestion DAG to populate the transactional database:

- Reads users.csv, products.csv, and orders.csv.
- Validates and cleans each record.
- Loads cleaned data into PostgreSQL tables.

14.7 Step 5 — Process Behavioural Data

Run the behavioural ingestion DAG:

```
| process_new_files_to_mongodb
```

This pipeline reads behavioural JSON files, validates each event, and stores valid records in the appropriate MongoDB collections.

14.8 Step 6 — Publish Transactional Data to Kafka

Trigger the postgres_to_kafka_batch_publish DAG to publish transactional records from PostgreSQL to the corresponding Kafka topics.

14.9 Step 7 — Publish Behavioural Data to Kafka

Trigger the mongo_to_kafka_batch_publish DAG to publish behavioural events from MongoDB to the corresponding Kafka topics.

14.10 Step 8 — Initialise ClickHouse

Create the required ClickHouse objects by executing the SQL scripts in the project:

- Kafka Engine tables (one per topic).
- Materialized Views (one per topic).
- Final MergeTree analytical tables.

After initialisation, ClickHouse will automatically begin consuming messages from Kafka.

14.11 Step 9 — Validate Data Flow

At this stage, the complete pipeline should be active. Verify the data flow using:

- Kafka UI — inspect topic contents and consumer group offsets.
- ClickHouse — run sample analytical queries to confirm data arrival.
- Airflow UI — review DAG execution history and task logs.

14.12 Step 10 — Access Dashboards

Once data is available in ClickHouse, connect Metabase to the ClickHouse database. Metabase can then be used to:

- Explore analytical datasets with an interactive query builder.
- Create and share business reports.
- Build KPI dashboards for stakeholder monitoring.